

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЁТ ПО ЗАДАНИЮ №6
«Сборка многомодульных программ. Вычисление корней
уравнений и определенных интегралов»

Вариант: 4 / 1 / 3

Выполнил:
студент 102 группы
Лукашев Д. А.

Преподаватели:
Калинин Г. М.
Цыбров Е. Г.

Москва
май, 2026

Содержание

Постановка задачи	2
Математическое описание	2
Уравнения кривых	2
Вычисление точек пересечения	2
Вычисление площади фигуры	2
Архитектура проекта и реализация	3
Структура проекта	3
Реализация функций на nasm x86 с fpu x87	3
Реализация функции <code>root</code>	4
Реализация функции <code>integral</code>	4
Вычисление площади фигуры	5
Возникшие трудности	5
Инструкция по запуску	6

Постановка задачи

В данной работе решается задача вычисления площади плоской фигуры, ограниченной тремя заданными кривыми. Программа должна самостоятельно с заданной точностью $\varepsilon_1 = 0.001$ найти абсциссы точек пересечения кривых, а затем вычислить площадь фигуры как алгебраическую сумму определенных интегралов с точностью $\varepsilon_2 = 0.001$.

Для выполнения задачи разработана многомодульная программа, состоящая из вычислительного модуля на языке Ассемблера, модуля численных методов на языке Си и главного файла, обеспечивающего парсинг аргументов командной строки и форматированный вывод. Сборка осуществляется с помощью утилиты `make`.

Математическое описание

Уравнения кривых

Согласно варианту 4, заданы следующие функции:

1. $f_1(x) = e^x + 2$
2. $f_2(x) = -\frac{1}{x}$
3. $f_3(x) = -\frac{2(x+1)}{3}$

Вычисление точек пересечения

Для нахождения абсцисс точек пересечения используется *метод деления отрезка пополам*. Данный метод ищет корень уравнения $f(x) - g(x) = 0$ на заданном отрезке $[a, b]$, сужая его границы в два раза на каждой итерации. На основе предварительного графического анализа были определены отрезки изоляции корней, ограничивающие заданную фигуру:

- x_1 (пересечение f_1 и f_3): отрезок $[-5.0, -3.0]$
- x_2 (пересечение f_3 и f_2): отрезок $[-2.5, -1.0]$
- x_3 (пересечение f_1 и f_2): отрезок $[-1.0, -0.1]$

Вычисление площади фигуры

Для вычисления определенных интегралов используется *формула Симпсона* (метод парабол) с автоматическим подбором шага интегрирования по правилу Рунге. Искомая площадь вычисляется как разность площадей под верхней и нижними кривыми на соответствующих отрезках интегрирования:

$$S = \int_{x_1}^{x_3} f_1(x)dx - \left(\int_{x_1}^{x_2} f_3(x)dx + \int_{x_2}^{x_3} f_2(x)dx \right)$$

Архитектура проекта и реализация

Сборка программы осуществляется утилитой `make` из нескольких модулей. Разделение проекта на файлы соответствует постановке задачи: функции f_1 , f_2 , f_3 реализуются на языке Ассемблера, а численные методы нахождения корней и вычисления интегралов — на языке Си.

Структура проекта

Программа состоит из следующих основных файлов:

- **functions.asm** — ассемблерный модуль, содержащий реализацию функций $f_1(x)$, $f_2(x)$, $f_3(x)$ с использованием математического сопроцессора x87.
- **functions.h** — заголовочный файл с объявлениями ассемблерных функций для их вызова из Си-кода.
- **methods.c** — модуль численных методов, содержащий функции `root` и `integral`.
- **methods.h** — заголовочный файл с прототипами численных методов и вспомогательных глобальных данных.
- **main.c** — главный модуль программы, выполняющий поиск точек пересечения, вычисление площади и обработку аргументов командной строки.
- **Makefile** — сценарий автоматической сборки программы и отчёта.

Реализация функций на `nasm` x86 с `fpu` x87

В соответствии с условием задания функции f_1 , f_2 , f_3 реализованы на языке Ассемблера с соглашением вызова `cdecl`. Аргумент типа `double` передаётся через стек, а результат возвращается на вершине стека сопроцессора x87.

Функция $f_1(x) = e^x + 2$ оказалась наиболее сложной в реализации. Сопроцессор x87 не содержит отдельной инструкции для прямого вычисления e^x , поэтому использовалось преобразование

$$e^x = 2^{x \log_2 e}.$$

После вычисления величины $x \log_2 e$ она разбивалась на целую и дробную части. Дробная часть вычислялась с помощью инструкции `fprem`, затем для неё применялась инструкция `f2xm1`, способная вычислять выражение вида $2^t - 1$ только при $t \in [-1, 1]$. После этого результат масштабировался инструкцией `fscale` с использованием целой части показателя степени. На последнем этапе к полученному значению e^x прибавлялась константа 2.

Функция $f_2(x) = -\frac{1}{x}$ реализована заметно проще. Для её вычисления в стек сопроцессора загружались значение 1.0 и аргумент x , после чего выполнялось

деление в правильном порядке. Отдельное внимание потребовалось уделить порядку загрузки операндов в стек `x87`, поскольку ошибка в расположении вершины стека приводила бы к вычислению $\frac{x}{1}$ вместо $\frac{1}{x}$. После деления знак результата менялся инструкцией `fchs`.

Функция $f_3(x) = -\frac{2(x+1)}{3}$ является линейной, поэтому её реализация оказалась самой короткой. В сопроцессор загружался аргумент x , затем последовательно выполнялись прибавление единицы, умножение на -2 и деление на 3 . Для этой функции основной сложностью было не само вычисление, а аккуратная организация констант и контроль за состоянием стека `x87`.

При реализации всех трёх функций существенную роль играла правильная работа со стеком сопроцессора. Промежуточные значения необходимо было своевременно удалять, чтобы не допустить переполнения стека `x87` и искажения результата при последующих вызовах функций.

Реализация функции `root`

Функция `root` реализована на языке Си и предназначена для нахождения абсциссы точки пересечения двух кривых. Для этого решается уравнение

$$f(x) = g(x),$$

которое преобразуется к виду

$$F(x) = f(x) - g(x) = 0.$$

В соответствии с вариантом задания для поиска корней использован метод деления отрезка пополам. На каждом шаге выбирается середина текущего отрезка $[a, b]$, после чего определяется, на какой из половин сохраняется смена знака функции $F(x)$. Затем рассматриваемый отрезок заменяется соответствующей половиной. Процесс продолжается до тех пор, пока длина отрезка не станет меньше заданной точности ε_1 .

Основная сложность при реализации функции `root` состояла не в самом методе, а в правильном выборе начальных отрезков изоляции корней. Метод дихотомии работает только тогда, когда на концах отрезка функция $F(x)$ имеет разные знаки. Поэтому перед программной реализацией был выполнен предварительный графический анализ, по которому вручную были подобраны интервалы для поиска всех трёх точек пересечения.

Дополнительно в реализации учитывалось число итераций, затраченных на поиск каждого корня. Это позволило выполнить одно из требований задания и вывести число шагов метода по запросу пользователя.

Реализация функции `integral`

Функция `integral` также реализована на языке Си. Она вычисляет определённый интеграл функции $f(x)$ на отрезке $[a, b]$ с заданной точностью ε_2 .

Согласно варианту задания для численного интегрирования использована формула Симпсона. Отрезок $[a, b]$ разбивается на чётное число частей, после чего интеграл приближённо вычисляется как взвешенная сумма значений функ-

ции в узлах разбиения. Для повышения точности число отрезков разбиения последовательно увеличивается.

Для автоматического подбора шага использовалось правило Рунге: сравнивались два последовательных приближения интеграла, полученных при разных значениях шага. Если разность между ними становилась достаточно малой, вычисление завершалось. Такой подход позволил получить требуемую точность без жёстко заданного количества разбиений.

Основная трудность здесь заключалась в корректной работе с пределами интегрирования. Поскольку площадь фигуры представлялась как разность нескольких интегралов, важно было правильно располагать пределы слева направо по оси x . Ошибка в порядке пределов приводила к неверному знаку интеграла и, как следствие, к ошибке в итоговой площади.

Вычисление площади фигуры

После нахождения трёх точек пересечения программа вычисляет площадь фигуры как комбинацию трёх определённых интегралов. Верхней границей фигуры на всём рассматриваемом промежутке служит функция $f_1(x)$, а нижняя граница задаётся функцией $f_3(x)$ на левом участке и функцией $f_2(x)$ на правом участке. Поэтому площадь вычисляется по формуле

$$S = \int_{x_1}^{x_3} f_1(x) dx - \left(\int_{x_1}^{x_2} f_3(x) dx + \int_{x_2}^{x_3} f_2(x) dx \right).$$

Такое разбиение было получено после анализа взаимного расположения графиков и проверки найденных точек пересечения.

Возникшие трудности

В ходе реализации были встречены следующие основные трудности:

- отсутствие прямой инструкции вычисления e^x в x87 и необходимость использовать представление через $2^{x \log_2 e}$;
- необходимость строго контролировать порядок операндов и состояние стека сопроцессора x87;
- подбор корректных интервалов изоляции корней для метода дихотомии;
- правильный выбор пределов интегрирования при вычислении площади;
- согласование точностей ε_1 и ε_2 так, чтобы итоговая площадь вычислялась с требуемой точностью.

В результате была получена многомодульная программа, в которой ассемблерный модуль отвечает за вычисление значений функций, а модули на языке Си — за реализацию численных методов и организацию пользовательского интерфейса.

Инструкция по запуску

Сборка проекта осуществляется с помощью утилиты автоматизации `make`. В корневой директории доступны следующие команды:

- `make` или `make all` — полная компиляция исходных кодов на Си и Ассемблере с последующей линковкой в исполняемый файл `program`, а также генерация данного отчета.
- `make clean` — удаление всех промежуточных объектных файлов (`.o`), временных файлов `LaTeX` и собранного бинарного файла.

Запуск программы осуществляется командой `./program` из терминала. По умолчанию программа вычисляет и выводит только итоговую площадь фигуры.

Для получения дополнительной информации предусмотрены флаги командной строки:

- `--help` или `-h`: Вывод справки. Программа напечатает список всех доступных команд и завершит свою работу без вычислений.
- `--roots` или `-r`: Печать найденных координат. Перед выводом площади программа отобразит рассчитанные абсциссы точек пересечения заданных кривых (x_1, x_2, x_3) .
- `--iters` или `-i`: Печать количества итераций. Программа выведет, сколько шагов цикла потребовалось методу деления отрезка пополам для локализации каждого из корней с заданной точностью.
- `--test root <f_id> <g_id> <a> <b> <eps>`: Запуск функции в режиме модульного тестирования. Программа найдет корень уравнения $f(x) = g(x)$ на отрезке $[a, b]$ с точностью eps , где `<f_id>` и `<g_id>` — порядковые номера функций (1, 2 или 3).
- `--test int <f_id> <a> <b> <eps>`: Запуск функции интегрирования в режиме модульного тестирования. Программа вычислит интеграл от функции `<f_id>` на отрезке $[a, b]$ с точностью eps .